

Using Data Manipulation Functions

In this chapter, you'll learn what functions are, what types of functions MySQL supports, and how to use these functions.

Understanding Functions

Like almost any other computer language, SQL supports the use of functions to manipulate data. Functions are operations that are usually performed on data, typically to facilitate conversion and manipulation.

An example of a function is the `RTrim()` function that we used in the last chapter to trim any spaces from the end of a string.

Note

Functions Are Less Portable Than SQL Code that runs on multiple systems is said to be *portable*. Most SQL statements are relatively portable, and when differences between SQL implementations occur, they are usually not very difficult to deal with. Functions, on the other hand, tend to be far less portable. Just about every major database management system (DBMS) supports functions that other DBMSs don't, and sometimes the differences are significant.

With code portability in mind, many SQL programmers opt not to use any implementation-specific features. Although this is a somewhat noble and idealistic view, it is not always in the best interests of application performance. If you opt not to use implementation-specific functions, you make your application code work harder. It must use other methods to do what the DBMS could have done more efficiently.

If you decide to use implementation-specific functions, make sure you comment your code well, so that at a later date you (or another developer) will know exactly to which SQL implementation you were writing.

Using Functions

Most SQL implementations support the following types of functions:

- **Text:** These functions are used to manipulate strings of text (for example, trimming or padding values and converting values to upper- and lowercase).
- **Numeric:** These functions are used to perform mathematical operations on numeric data (for example, returning absolute numbers and performing algebraic calculations).
- **Date and time:** These functions are used to manipulate date and time values and to extract specific components from these values (for example, returning differences between dates and checking date validity).
- **System:** These functions return information specific to the DBMS being used (for example, returning user login information or checking version specifics).

Text Manipulation Functions

You’ve already seen an example of text-manipulation functions in the last chapter: You saw how to use the `RTrim()` function to trim white space from the end of a column value. Here is another example, this time using the `Upper()` function:

► Input

```
SELECT vend_name, Upper(vend_name) AS vend_name_upcase
FROM vendors
ORDER BY vend_name;
```

► Output

+-----+-----+	
vend_name	vend_name_upcase
+-----+-----+	
ACME	ACME
Anvils R Us	ANVILS R US
Furball Inc.	FURBALL INC.
Jet Set	JET SET
Jouets Et Ours	JOUETS ET OURS
LT Supplies	LT SUPPLIES
+-----+-----+	

► Analysis

As you can see, `Upper()` converts text to uppercase, and so in this example, each vendor is listed twice: The first time it is listed exactly as it is stored in the `vendors` table, and then it is converted to uppercase in the column `vend_name_upcase`.

Table 11.1 lists some commonly used text-manipulation functions.

Table 11.1 Commonly Used Text-Manipulation Functions

Function	Description
Left()	Returns characters from the left of a string
Length()	Returns the length of a string
Locate()	Finds a substring within a string
Lower()	Converts a string to lowercase
LTrim()	Trims white space from the left of a string
Right()	Returns characters from the right of a string
RTrim()	Trims white space from the right of a string
Soundex()	Returns a string's SOUNDEX value
SubString()	Returns characters from within a string
Upper()	Converts a string to uppercase

One item in Table 11.1 requires further explanation. SOUNDEX is an algorithm that converts any string of text into an alphanumeric pattern that describes the phonetic representation of that text. SOUNDEX takes into account similar-sounding characters and syllables to enable strings to be compared based on how they sound rather than how they have been typed. Although SOUNDEX is not a SQL concept, MySQL (like many other DBMSs) offers SOUNDEX support.

Here's an example of using the Soundex() function. Customer Coyote Inc. is in the customers table and has a contact named Y. Lee. But what if that is a typo, and the contact is actually supposed to be Y. Lie? Obviously, searching by the correct contact name will return no data, as shown here:

► Input

```
SELECT cust_name, cust_contact
FROM customers
WHERE cust_contact = 'Y. Lie';
```

► Output

```
+-----+-----+
| cust_name | cust_contact |
+-----+-----+
```

Now try the same search using the Soundex() function to match all contact names that sound similar to Y. Lie:

► Input

```
SELECT cust_name, cust_contact
FROM customers
WHERE Soundex(cust_contact) = Soundex('Y Lie');
```

► Output

+	-----+	-----+
	cust_name	cust_contact
+	-----+	-----+
	Coyote Inc.	Y Lee
+	-----+	-----+

► Analysis

In this example, the WHERE clause uses the Soundex() function to convert both the cust_contact column value and the search string to their SOUNDEX values. Because Y. Lee and Y. Lie sound alike, their SOUNDEX values match, and so the WHERE clause correctly filters the desired data.

Note

Understanding SOUNDEX In case you are curious about how the Soundex() function works its magic, I'll tell you the secret: SOUNDEX is a formula that converts any string of text into a four-character value that represents its sound. The WHERE statement used above simply obtains the SOUNDEX value for Y Lie and compares it to the SOUNDEX value for every contact in the table:

```
WHERE Soundex(cust_contact) = Soundex('Y Lie');
```

If you're interested, you can use this SQL statement to see every contact and its SOUNDEX value:

```
SELECT cust_contact,
       SOUNDEX(cust_contact) AS cust_contact_soundex
FROM customers
```

Date and Time Manipulation Functions

Dates and times are stored in tables using special datatypes with special internal formats so they can be sorted or filtered quickly and efficiently and in order to save physical storage space.

The format used to store dates and times is usually of no use to your applications, and so date and time functions are almost always used to read, expand, and manipulate these values. Because of this, date and time manipulation functions are some of the most important functions in the MySQL language.

Table 11.2 lists some commonly used date and time manipulation functions.

Table 11.2 Commonly Used Date and Time Manipulation Functions

Function	Description
AddDate()	Adds to a date (days, weeks, and so on)
AddTime()	Adds to a time (hours, minutes, and so on)

Function	Description
<code>CurDate()</code>	Returns the current date
<code>CurTime()</code>	Returns the current time
<code>Date()</code>	Returns the date portion of a datetime value
<code>DateDiff()</code>	Calculates the difference between two dates
<code>Date_Add()</code>	Does date arithmetic
<code>Date_Format()</code>	Returns a formatted date or time string
<code>Day()</code>	Returns the day portion of a date
<code>DayOfWeek()</code>	Returns the day of week for a date
<code>Hour()</code>	Returns the hour portion of a time
<code>Minute()</code>	Returns the minute portion of a time
<code>Month()</code>	Returns the month portion of a date
<code>Now()</code>	Returns the current date and time
<code>Second()</code>	Returns the second portion of a time
<code>Time()</code>	Returns the time portion of a datetime value
<code>Year()</code>	Returns the year portion of a date

This would be a good time to revisit data filtering using `WHERE`. Thus far we have filtered data by using `WHERE` clauses that compare numbers and text, but you frequently need to filter data by date. Filtering by date requires some extra care and the use of special MySQL functions.

The first thing to keep in mind is the date format used by MySQL. Whenever you specify a date, whether to insert or update table values or to filter using a `WHERE` clause, the date must be in the format `yyyy-mm-dd`. So, for September 1, 2023, you specify `2023-09-01`. Although other date formats might be recognized, this is the preferred date format because it eliminates ambiguity. (Is `04/05/06` May 4, 2006, or April 5, 2006, or May 6, 2004?)

Tip

Always Use Four-Digit Years MySQL supports two-digit years. For example, it treats the years 00–69 as 2000–2069 and the years 70–99 as 1970–1999. While these might in fact be the intended years, it is far safer to always use a four-digit year so MySQL does not have to make any assumptions for you.

A basic date comparison is simple enough:

► Input

```
SELECT cust_id, order_num
FROM orders
WHERE order_date = '2023-09-01';
```

► Output

```
+-----+-----+
| cust_id | order_num |
+-----+-----+
| 10001 | 20005 |
+-----+-----+
```

► Analysis

The `SELECT` statement worked in this case. It retrieved a single order record—one with an `order_date` of 2023-09-01.

But is using `WHERE order_date = '2023-09-01'` safe? `order_date` has the `datetime` datatype. This type stores dates along with time values. The values in our example tables all have times of 00:00:00, but that might not always be the case. What if order dates are stored using the current date and time (so you not only know the order date but also the time of day the order was placed)? In this case, `WHERE order_date = '2023-09-01'` fails if, for example, the stored `order_date` value is 2023-09-01 11:30:05. Even though a row with that date is present, it is not retrieved because the `WHERE` match fails.

The solution is to instruct MySQL to only compare the specified date to the date portion of the column instead of using the entire column value. To do this, you must use the `Date()` function. `Date(order_date)` instructs MySQL to extract just the date part of the column, and so a safer `SELECT` statement is as follows:

► Input

```
SELECT cust_id, order_num
FROM orders
WHERE Date(order_date) = '2023-09-01';
```

Tip

If You Mean Date, Use `Date()` It's a good practice to use `Date()` if what you want is just the date, even if you know that the column contains only dates. This way, if a date-time value somehow ends up in the table in the future, your SQL won't break. Oh, and yes, there is a `Time()` function, too, and you should use it when you want the time.

Now that you know how to use dates to test for equality, using all of the other operators (introduced in Chapter 6, “Filtering Data”) should be self-explanatory.

But one other type of date comparison warrants explanation. What if you wanted to retrieve all orders placed in September 2023? A simple equality test does not work as it matches the day of month, too. There are several solutions, one of which follows:

► Input

```
SELECT cust_id, order_num
FROM orders
WHERE Date(order_date) BETWEEN '2023-09-01'
AND '2023-09-30';
```

► Output

+-----+	
cust_id	order_num
+-----+	
10001	20005
10003	20006
10004	20007
+-----+	

► Analysis

Here a `BETWEEN` operator is used to define 2023-09-01 and 2023-09-30 as the range of dates to match.

Here's another solution (one that won't require you to remember how many days are in each month or worry about February in leap years):

► Input

```
SELECT cust_id, order_num
FROM orders
WHERE Year(order_date) = 2023
AND Month(order_date) = 9;
```

► Analysis

`Year()` is a function that returns the year from a date (or a datetime). Similarly, `Month()` returns the month from a date. `WHERE Year(order_date) = 2023 AND Month(order_date) = 9` thus retrieves all rows that have an `order_date` in year 2023 and in month 9.

Tip

How to Ignore Time Using functions like `Year()`, `Month()`, and `Day()` allows you to work with dates without worrying about any time values that may be stored with them.

Numeric Manipulation Functions

Numeric manipulation functions do what it sounds like they do: manipulate numeric data. These functions tend to be used primarily for algebraic, trigonometric, or geometric calculations and, therefore, are not as frequently used as string or date and time manipulation functions.

The ironic thing is that of all the functions found in the major DBMSs, the numeric functions are the ones that are most uniform and consistent. Table 11.3 lists some of the most commonly used numeric manipulation functions.

Table 11.3 Commonly Used Numeric Manipulation Functions

Function	Description
Abs ()	Returns the absolute value of a specified number
Cos ()	Returns the trigonometric cosine of a specified angle
Exp ()	Returns the exponential value of a specified number
Mod ()	Returns the remainder of a division operation
Pi ()	Returns the value of pi
Rand ()	Returns a random number
Sin ()	Returns the trigonometric sine of a specified angle
Sqrt ()	Returns the square root of a specified number
Tan ()	Returns the trigonometric tangent of a specified angle

Summary

In this chapter, you learned how to use SQL's data manipulation functions and paid special attention to working with dates.

Challenges

1. Our store is now online, and customer accounts are being created. Every user needs a login, and the default login will be a combination of a user's name and city. Write a SQL statement that returns customer ID (`cust_id`), customer name (`customer_name`), and `user_login`, which is all uppercase and composed of the first two characters of the customer contact (`cust_contact`) and the first three characters of the customer city (`cust_city`). So, for example, my login (Ben Forta, living in Oak Park) would be BE0AK. Here's a hint: For this one, you'll use functions, concatenation, and an alias.
2. Write a SQL statement to return the order number (`order_num`) and order date (`order_date`) for every order placed in October 2023 and sort the list by order date.